Low precision floating point types

Contents

- FP4 (4-bit Precision)
- FP6 (6-bit Precision)
- FP8 (Quarter Precision)
- Float16 (Half Precision)
- BFloat16 (Brain float 16-bit precision)
- HIP Extensions
- hipExt Types
- C-APIs
- HIP EXT C++ API

Modern computing tasks often require balancing numerical precision against hardware resources and processing speed. Low precision floating point number formats in HIP include FP8 (Quarter Precision) and FP16 (Half Precision), which reduce memory and bandwidth requirements compared to traditional 32-bit or 64-bit formats. The following sections detail their specifications, variants, and provide practical guidance for implementation in HIP.

FP4 (4-bit Precision)

docs-7.2.1

FP4 (Floating Point 4-bit) numbers represent the current extreme pushing the boundaries of memory optimization for specialized AI workloads. This ultra-compact format is designed for scenarios where model size and computational efficiency are

paramount constraints, even at the cost of significant precision reduction.

FP4 is particularly valuable in weight storage for large language models (LLMs) and vision transformers, where aggressive quantization can dramatically reduce model size while maintaining acceptable inference quality. By reducing memory footprint to a quarter of FP16, FP4 enables deployment of larger models in memory-constrained environments or higher throughput in existing hardware.

The supported FP4 format is:

- **E2M1 Format**
  - Sign: 1 bit
  - Exponent: 2 bits
  - Mantissa: 1 bit

The E2M1 format offers a balance between minimal precision and a reasonable dynamic range, optimized for weight storage in neural network applications.

HIP Header

The HIP FP4 header defines the FP4 numbers.

Device Compatibility

The following table shows hardware support for this precision format by GPU architecture. “Yes” indicates native hardware acceleration is available, while “No” indicates hardware acceleration is not available.

Device Type	E2M1
CDNA1	No
CDNA2	No
CDNA3	No
CDNA4	Yes
RDNA2	No
RDNA3	No
RDNA4	No

Using FP4 Numbers in HIP Programs

To use the FP4 numbers inside HIP programs:

```
#include <hip/hip_fp4.h>
```

FP4 numbers can be used on CPU side:

```
__hip_fp4_storage_t convert_float_to_fp4(
  float in, /* Input val */
  __hip_saturation_t sat /* Saturation behavior */
  ) {
  return __hip_cvt_float_to_fp4(in, __HIP_E2M1, sat);
}
```

The same can be done in kernels as well:

```
__device__ __hip_fp4_storage_t d_convert_float_to_fp4(
  float in,
  __hip_saturation_t sat) {
  return __hip_cvt_float_to_fp4(in, __HIP_E2M1, sat);
}
```

docs-7.2.1

The following code example demonstrates a simple roundtrip conversion using FP4 types:

```
#include <hip/hip_fp4.h>
#include <hip/hip_runtime.h>
#include <iostream>
#include <vector>

#define hip_check(hip_call)
{
    auto hip_res = hip_call;
    if (hip_res != hipSuccess) {
      std::cerr << "Failed in HIP call: " << #hip_call \
                << " at " << __FILE__ << ":" << __LINE__ \
                << " with error: " << hipGetErrorString(hip_res) << std::end
      std::abort();
    }
}

__global__ void float_to_fp4_to_float(float *in,
                                      __hip_saturation_t sat, float *out,
                                      size_t size) {
    int i = threadIdx.x;
    if (i < size) {
        auto fp4 = __hip_cvt_float_to_fp4(in[i], __HIP_E2M1, sat);
        out[i] = __hip_cvt_fp4_to_halfraw(fp4, __HIP_E2M1);
    }
}

int main() {
    constexpr size_t size = 16;
    hipDeviceProp_t prop;
    hip_check(hipGetDeviceProperties(&prop, 0));
    bool is_supported = (std::string(prop.gcnArchName).find("gfx950") != std
    if(!is_supported) {
        std::cerr << "Need gfx950, but found: " << prop.gcnArchName << std::
        std::cerr << "Device conversions are not supported on this hardware.
        return -1;
    }

    constexpr __hip_saturation_t sat = __HIP_SATFINITE;

    // Create test data
    std::vector<float> in;
    in.reserve(size);
    for (size_t i = 0; i < size; i++) {
        in.push_back(i * 0.5f);
    }

    // Allocate device memory
    float *d_in, *d_out;
    hip_check(hipMalloc(&d_in, sizeof(float) * size))·
    hip_check(hipMalloc(&d_out, sizeof(float) * size)
    hip_check(hipMemcpy(d_in, in.data(), sizeof(float

    // Run conversion kernel
```

docs-7.2.1

```
    float_to_fp4_to_float<<<1, size>>>(d_in, sat, d_out, size);

    // Get results
    std::vector<float> result(size);
    hip_check(hipMemcpy(result.data(), d_out, sizeof(float) * size, hipMemcp

    // Clean up
    hip_check(hipFree(d_in));
    hip_check(hipFree(d_out));

    // Display results
    std::cout << "FP4 Roundtrip Results:" << std::endl;
    for (size_t i = 0; i < size; i++) {
        std::cout << "Original: " << in[i] << " -> FP4 roundtrip: " << resul
    }

    return 0;
}
```

There are C++ style classes available as well:

```
__hip_fp4_e2m1 fp4_val(1.0f);
```

FP4 type has its own class:

- `__hip_fp4_e2m1`

There is support of vector of FP4 types:

- `__hip_fp4x2_e2m1`: holds 2 values of FP4 e2m1 numbers
- `__hip_fp4x4_e2m1`: holds 4 values of FP4 e2m1 numbers

FP6 (6-bit Precision)

FP6 (Floating Point 6-bit) numbers represent an even more aggressive memory optimization compared to FP8, designed specifically for ultra-efficient deep learning inference and specialized AI applications. This extremely compact format delivers significant memory and bandwidth savings at the cost of reduced dynamic range and precision.

The primary advantage of FP6 is enabling higher computational th
constrained environments, particularly for AI model deployment on edge devices and
applications where model size is a critical constraint. While offering less precision than FP8,

docs-7.2.1

FP6 maintains sufficient accuracy for many inference tasks, especially when used with carefully quantized models.

There are two primary FP6 formats:

- **E3M2 Format**
  - Sign: 1 bit
  - Exponent: 3 bits
  - Mantissa: 2 bits
- **E2M3 Format**
  - Sign: 1 bit
  - Exponent: 2 bits
  - Mantissa: 3 bits

The E3M2 format provides a wider numeric range with less precision, while the E2M3 format offers higher precision within a narrower range.

HIP Header

The HIP FP6 header defines the FP6 numbers.

Device Compatibility

The following table shows hardware support for this precision format by GPU architecture. “Yes” indicates native hardware acceleration is available, while “No” indicates hardware acceleration is not available.

Device Type	E3M2	E2M3
CDNA1	No	No
CDNA2	No	No
CDNA3	No	No
CDNA4	Yes	Yes
RDNA2	No	No
RDNA3	No	No
RDNA4	No	No

Using FP6 Numbers in HIP Programs

To use the FP6 numbers inside HIP programs:

```
#include <hip/hip_fp6.h>
```

FP6 numbers can be used on CPU side:

```
__hip_fp6_storage_t convert_float_to_fp6(
  float in, /* Input val */
  __hip_fp6_interpretation_t interpret, /* interpretation of number E3M2/E2M
  __hip_saturation_t sat /* Saturation behavior */
  ) {
  return __hip_cvt_float_to_fp6(in, interpret, sat);
}
```

The same can be done in kernels as well:

```
__device__ __hip_fp6_storage_t d_convert_float_to_fp6(
  float in,
  __hip_fp6_interpretation_t interpret,
  __hip_saturation_t sat) {
  return __hip_cvt_float_to_fp6(in, interpret, sat);
}
```

The following code example demonstrates a roundtrip conversion using FP6 types:

```
#include <hip/hip_fp6.h>
#include <hip/hip_runtime.h>
#include <iostream>
#include <vector>

#define hip_check(hip_call)
{
    auto hip_res = hip_call;
    if (hip_res != hipSuccess) {
      std::cerr << "Failed in HIP call: " << #hip_call \
                << " at " << __FILE__ << ":" << __LINE__ \
                << " with error: " << hipGetErrorString(hip_res) << std::end
      std::abort();
    }
}

__global__ void float_to_fp6_to_float(float *in,
                                      __hip_fp6_interpretation_t interpret,
                                      __hip_saturation_t sat, float *out,
                                      size_t size) {
    int i = threadIdx.x;
    if (i < size) {
        auto fp6 = __hip_cvt_float_to_fp6(in[i], interpret, sat);
        out[i] = __hip_cvt_fp6_to_halfraw(fp6, interpret);
    }
}

int main() {
    constexpr size_t size = 16;
    hipDeviceProp_t prop;
    hip_check(hipGetDeviceProperties(&prop, 0));
    bool is_supported = (std::string(prop.gcnArchName).find("gfx950") != std
    if(!is_supported) {
        std::cerr << "Need gfx950, but found: " << prop.gcnArchName << std::
        std::cerr << "Device conversions are not supported on this hardware.
        return -1;
    }

    // Test both formats
    const __hip_saturation_t sat = __HIP_SATFINITE;

    // Create test vectors
    std::vector<float> in(size);
    for (size_t i = 0; i < size; i++) {
        in[i] = i * 0.5f;
    }

    std::vector<float> out_e2m3(size);
    std::vector<float> out_e3m2(size);

    // Allocate device memory
    float *d_in, *d_out;
    hip_check(hipMalloc(&d_in, sizeof(float) * size));
```

docs-7.2.1

```
    hip_check(hipMalloc(&d_out, sizeof(float) * size));
    hip_check(hipMemcpy(d_in, in.data(), sizeof(float) * size, hipMemcpyHost

    // Test E2M3 format
    float_to_fp6_to_float<<<1, size>>>(d_in, __HIP_E2M3, sat, d_out, size);
    hip_check(hipMemcpy(out_e2m3.data(), d_out, sizeof(float) * size, hipMem

    // Test E3M2 format
    float_to_fp6_to_float<<<1, size>>>(d_in, __HIP_E3M2, sat, d_out, size);
    hip_check(hipMemcpy(out_e3m2.data(), d_out, sizeof(float) * size, hipMem

    // Display results
    std::cout << "FP6 Roundtrip Results:" << std::endl;
    for (size_t i = 0; i < size; i++) {
        std::cout << "Original: " << in[i]
                  << " -> E2M3: " << out_e2m3[i]
                  << " -> E3M2: " << out_e3m2[i] << std::endl;
    }

    // Clean up
    hip_check(hipFree(d_in));
    hip_check(hipFree(d_out));

    return 0;
}
```

There are C++ style classes available as well:

```
__hip_fp6_e2m3 fp6_val_e2m3(1.1f);
__hip_fp6_e3m2 fp6_val_e3m2(1.1f);
```

Each type of FP6 number has its own class:

- `__hip_fp6_e2m3`
- `__hip_fp6_e3m2`

There is support of vector of FP6 types:

- `__hip_fp6x2_e2m3`: holds 2 values of FP6 e2m3 numbers
- `__hip_fp6x4_e2m3`: holds 4 values of FP6 e2m3 numbers
- `__hip_fp6x2_e3m2`: holds 2 values of FP6 e3m2 numbers
- `__hip_fp6x4_e3m2`: holds 4 values of FP6 e3m2 numbers

docs-7.2.1

FP8 (Quarter Precision)

FP8 (Floating Point 8-bit) numbers were introduced as a compact numerical format specifically tailored for deep learning inference. By reducing precision while maintaining computational effectiveness, FP8 allows for significant memory savings and improved processing speed. This makes it particularly beneficial for deploying large-scale models with strict efficiency constraints.

Unlike traditional floating-point formats such as FP32 or even FP16, FP8 further optimizes performance by enabling a higher volume of matrix operations per second. Its reduced bit-width minimizes bandwidth requirements, making it an attractive choice for hardware accelerators in deep learning applications.

There are two primary FP8 formats:

- **E4M3 Format**
  - Sign: 1 bit
  - Exponent: 4 bits
  - Mantissa: 3 bits
- **E5M2 Format**
  - Sign: 1 bit
  - Exponent: 5 bits
  - Mantissa: 2 bits

The E4M3 format offers higher precision with a narrower range, while the E5M2 format provides a wider range at the cost of some precision.

Additionally, FP8 numbers have two representations:

- **FP8-OCP (Open Compute Project)**
  - This is a standardized format developed by the Open Compute Project to ensure compatibility across various hardware and software implementations.
- **FP8-FNUZ (Finite and NaN Only)**
  - A specialized format optimized for specific computations, NaN values (no Inf support).
  - This provides one extra value of exponent and adds to the range of supported FP8

docs-7.2.1

numbers.

- **NaN Definition**: When the sign bit is set, and all other exponent and mantissa bits are zero.

The FNUZ representation provides an extra exponent value, expanding the range of representable numbers compared to standard FP8 formats.

HIP Header

The HIP FP8 header defines the FP8 ocp/fnuz numbers.

Device Compatibility

The following table shows hardware support for this precision format by GPU architecture. “Yes” indicates native hardware acceleration is available, while “No” indicates hardware acceleration is not available.

Supported devices for fp8 numbers

Device Type	FNUZ FP8	OCP FP8
CDNA1	No	No
CDNA2	No	No
CDNA3	Yes	No
CDNA4	No	Yes
RDNA2	No	No
RDNA3	No	No
RDNA4	No	Yes

docs-7.2.1

Using FP8 Numbers in HIP Programs

To use the FP8 numbers inside HIP programs.

```
#include <hip/hip_fp8.h>
```

FP8 numbers can be used on CPU side:

```
__hip_fp8_storage_t convert_float_to_fp8(
  float in, /* Input val */
  __hip_fp8_interpretation_t interpret, /* interpretation of number E4M3/E5M
  __hip_saturation_t sat /* Saturation behavior */
  ) {
  return __hip_cvt_float_to_fp8(in, sat, interpret);
}
```

The same can be done in kernels as well.

```
__device__ __hip_fp8_storage_t d_convert_float_to_fp8(
  float in,
  __hip_fp8_interpretation_t interpret,
  __hip_saturation_t sat) {
  return __hip_cvt_float_to_fp8(in, sat, interpret);
}
```

Note: On a gfx94x GPU, the type will default to the fnuz type.

The following code example does roundtrip FP8 conversions on both the CPU and GPU and compares the results.

```
#include <hip/hip_fp8.h>
#include <hip/hip_runtime.h>
#include <iostream>
#include <vector>

#define hip_check(hip_call)
{
    auto hip_res = hip_call;
    if (hip_res != hipSuccess) {
      std::cerr << "Failed in HIP call: " << #hip_call \
                << " at " << __FILE__ << ":" << __LINE__ \
                << " with error: " << hipGetErrorString(hip_res) << std::end
      std::abort();
    }
}

__device__ __hip_fp8_storage_t d_convert_float_to_fp8(
    float in, __hip_fp8_interpretation_t interpret, __hip_saturation_t sat)
    return __hip_cvt_float_to_fp8(in, sat, interpret);
}

__device__ float d_convert_fp8_to_float(float in,
                                        __hip_fp8_interpretation_t interpret
    float hf = __hip_cvt_fp8_to_float(in, interpret);
    return hf;
}

__global__ void float_to_fp8_to_float(float *in,
                                      __hip_fp8_interpretation_t interpret,
                                      __hip_saturation_t sat, float *out,
                                      size_t size) {
    int i = threadIdx.x;
    if (i < size) {
        auto fp8 = d_convert_float_to_fp8(in[i], interpret, sat);
        out[i] = d_convert_fp8_to_float(fp8, interpret);
    }
}

__hip_fp8_storage_t
convert_float_to_fp8(float in, /* Input val */
                     __hip_fp8_interpretation_t
                         interpret, /* interpretation of number E4M3/E5M2 */
                     __hip_saturation_t sat /* Saturation behavior */
) {
    return __hip_cvt_float_to_fp8(in, sat, interpret);
}

float convert_fp8_to_float(
    __hip_fp8_storage_t in, /* Input val */
    __hip_fp8_interpretation_t
        interpret /* interpretation of number E4M3/E5
) {
    __half hf = __hip_cvt_fp8_to_halfraw(in, interpret);
```

docs-7.2.1

```
    return hf;
}

int main() {
    constexpr size_t size = 32;
    hipDeviceProp_t prop;
    hip_check(hipGetDeviceProperties(&prop, 0));
    bool is_supported = (std::string(prop.gcnArchName).find("gfx94") != std:
                        || (std::string(prop.gcnArchName).find("gfx950") !=
                        || (std::string(prop.gcnArchName).find("gfx12") != s
    if(!is_supported) {
        std::cerr << "Need a gfx94x, gfx950 or gfx12xx, but found: " << prop
        std::cerr << "No device conversions are supported, only host convers
        return -1;
    }

    const __hip_fp8_interpretation_t interpret = (std::string(prop.gcnArchNa
                                                  ? __HIP_E4M3_FNUZ // gfx
                                                  : __HIP_E4M3;
    constexpr __hip_saturation_t sat = __HIP_SATFINITE;

    std::vector<float> in;
    in.reserve(size);
    for (size_t i = 0; i < size; i++) {
        in.push_back(i + 1.1f);
    }

    std::cout << "Converting float to fp8 and back..." << std::endl;
    // CPU convert
    std::vector<float> cpu_out;
    cpu_out.reserve(size);
    for (const auto &fval : in) {
        auto fp8 = convert_float_to_fp8(fval, interpret, sat);
        cpu_out.push_back(convert_fp8_to_float(fp8, interpret));
    }

    // GPU convert
    float *d_in, *d_out;
    hip_check(hipMalloc(&d_in, sizeof(float) * size));
    hip_check(hipMalloc(&d_out, sizeof(float) * size));

    hip_check(hipMemcpy(d_in, in.data(), sizeof(float) * in.size(),
                        hipMemcpyHostToDevice));

    float_to_fp8_to_float<<<1, size>>>(d_in, interpret, sat, d_out, size);

    std::vector<float> gpu_out(size, 0.0f);
    hip_check(hipMemcpy(gpu_out.data(), d_out, sizeof(float) * gpu_out.size(
                        hipMemcpyDeviceToHost));

    hip_check(hipFree(d_in));
    hip_check(hipFree(d_out));

    // Validation
```

docs-7.2.1

```
    for (size_t i = 0; i < size; i++) {
        if (cpu_out[i] != gpu_out[i]) {
            std::cerr << "cpu round trip result: " << cpu_out[i]
                      << " - gpu round trip result: " << gpu_out[i] << std::
            std::abort();
        }
    }
    std::cout << "...CPU and GPU round trip convert matches." << std::endl;

    return 0;
}
```

There are C++ style classes available as well.

```
__hip_fp8_e4m3_fnuz fp8_val(1.1f); // gfx94x
__hip_fp8_e4m3 fp8_val(1.1f);
```

Each type of FP8 number has its own class:

- `__hip_fp8_e4m3`
- `__hip_fp8_e5m2`
- `__hip_fp8_e4m3_fnuz`
- `__hip_fp8_e5m2_fnuz`

There is support of vector of FP8 types.

- `__hip_fp8x2_e4m3`: holds 2 values of OCP FP8 e4m3 numbers
- `__hip_fp8x4_e4m3`: holds 4 values of OCP FP8 e4m3 numbers
- `__hip_fp8x2_e5m2`: holds 2 values of OCP FP8 e5m2 numbers
- `__hip_fp8x4_e5m2`: holds 4 values of OCP FP8 e5m2 numbers
- `__hip_fp8x2_e4m3_fnuz`: holds 2 values of FP8 fnuz e4m3 numbers
- `__hip_fp8x4_e4m3_fnuz`: holds 4 values of FP8 fnuz e4m3 numbers
- `__hip_fp8x2_e5m2_fnuz`: holds 2 values of FP8 fnuz e5m2 numbers
- `__hip_fp8x4_e5m2_fnuz`: holds 4 values of FP8 fnuz e5m2 numbers

FNUZ extensions will be available on gfx94x only.

Float16 (Half Precision)

`float16` (Floating Point 16-bit) numbers offer a balance between precision and efficiency, making them a widely adopted standard for accelerating deep learning inference. With higher precision than FP8 but lower memory requirements than FP32, `float16` enables faster computations while preserving model accuracy.

Deep learning workloads often involve massive datasets and complex calculations, making FP32 computationally expensive. `float16` helps mitigate these costs by reducing storage and bandwidth demands, allowing for increased throughput without significant loss of numerical stability. This format is particularly useful for training and inference in GPUs and TPUs optimized for half-precision arithmetic.

Float16 Format

The `float16` format uses the following bit allocation:

- **Sign**: 1 bit
- **Exponent**: 5 bits
- **Mantissa**: 10 bits

This format offers higher precision with a narrower range compared to `bfloat16`.

HIP Header

The HIP FP16 header defines the `float16` format.

Device Compatibility

This precision format is supported across all GPU architectures. The HIP types and functions are available for use in both host and device code, with implementation handled by the compiler and device libraries.

Using Float16 Numbers in HIP Progra

docs-7.2.1

To use `float16` numbers inside HIP programs:

```
#include <hip/hip_fp16.h> // for float16
```

The following code example adds two `float16` values on the GPU and compares the results against summed float values on the CPU.

```
#include <hip/hip_fp16.h>
#include <hip/hip_runtime.h>
#include <iostream>
#include <vector>

#define hip_check(hip_call)
{
    auto hip_res = hip_call;
    if (hip_res != hipSuccess) {
        std::cerr << "Failed in HIP call: " << #hip_call \
                  << " at " << __FILE__ << ":" << __LINE__ \
                  << " with error: " << hipGetErrorString(hip_res) << std::e
        std::abort();
    }
}

__global__ void add_half_precision(__half* in1, __half* in2, float* out, siz
    int idx = threadIdx.x;
    if (idx < size) {
        // Load as half, perform addition in float, store as float
        __half sum = in1[idx] + in2[idx];
        out[idx] = __half2float(sum);
    }
}

int main() {
    constexpr size_t size = 32;
    constexpr float tolerance = 1e-1f;  // Allowable numerical difference

    // Initialize input vectors as floats
    std::vector<float> in1(size), in2(size);
    for (size_t i = 0; i < size; i++) {
        in1[i] = i + 0.5f;
        in2[i] = i + 0.5f;
    }

    // Compute expected results in full precision on CPU
    std::vector<float> cpu_out(size);
    for (size_t i = 0; i < size; i++) {
        cpu_out[i] = in1[i] + in2[i];  // Direct float addition
    }

    // Allocate device memory (store input as half, output as float)
    __half *d_in1, *d_in2;
    float *d_out;
    hip_check(hipMalloc(&d_in1, sizeof(__half) * size));
    hip_check(hipMalloc(&d_in2, sizeof(__half) * size));
    hip_check(hipMalloc(&d_out, sizeof(float) * size));

    // Convert input to half and copy to device
    std::vector<__half> in1_half(size), in2_half(size
    for (size_t i = 0; i < size; i++) {
        in1_half[i] = __float2half(in1[i]);
```

docs-7.2.1

```
        in2_half[i] = __float2half(in2[i]);
    }

    hip_check(hipMemcpy(d_in1, in1_half.data(), sizeof(__half) * size, hipMe
    hip_check(hipMemcpy(d_in2, in2_half.data(), sizeof(__half) * size, hipMe

    // Launch kernel
    add_half_precision<<<1, size>>>(d_in1, d_in2, d_out, size);

    // Copy result back to host
    std::vector<float> gpu_out(size, 0.0f);
    hip_check(hipMemcpy(gpu_out.data(), d_out, sizeof(float) * size, hipMemc

    // Free device memory
    hip_check(hipFree(d_in1));
    hip_check(hipFree(d_in2));
    hip_check(hipFree(d_out));

    // Validation with tolerance
    for (size_t i = 0; i < size; i++) {
        if (std::fabs(cpu_out[i] - gpu_out[i]) > tolerance) {
            std::cerr << "Mismatch at index " << i << ": CPU result = " << c
                      << ", GPU result = " << gpu_out[i] << std::endl;
            std::abort();
        }
    }

    std::cout << "Success: CPU and GPU half-precision addition match within

    return 0;
}
```

C++ Style Classes

Float16 numbers can be used with C++ style classes:

```
__half fp16_val(1.1f);          // float16
```

Vector Support

There is support for vectors of float16 types:

- `__half2`: holds 2 values of float16 numbers

docs-7.2.1

BFloat16 (Brain float 16-bit precision)

`bfloat16` (Brain Floating Point 16-bit) is a truncated version of the 32-bit IEEE 754 single-precision floating-point format. Originally developed by Google for machine learning applications, `bfloat16` provides a good balance between range and precision for neural network computations.

`bfloat16` is particularly well-suited for deep learning workloads because it maintains the same exponent range as FP32, making it less prone to overflow and underflow issues during training. This format sacrifices some precision compared to float16 but offers better numerical stability for many AI applications.

BFloat16 Format

The `bfloat16` format uses the following bit allocation:

- **Sign**: 1 bit
- **Exponent**: 8 bits
- **Mantissa**: 7 bits

This format provides a wider range at the cost of some precision compared to `float16`.

HIP Header

The HIP BF16 header defines the `bfloat16` format.

Device Compatibility

This precision format is supported across all GPU architectures. The HIP types and functions are available for use in both host and device code, with implementation handled by the compiler and device libraries.

Using `bfloat16` Numbers in HIP Pro

docs-7.2.1

To use `bfloat16` numbers inside HIP programs:

```
#include <hip/hip_bf16.h> // for bfloat16
```

The following code example demonstrates basic `bfloat16` operations:

```
#include <hip/hip_bf16.h>
#include <hip/hip_runtime.h>
#include <iostream>
#include <vector>

#define hip_check(hip_call)
{
    auto hip_res = hip_call;
    if (hip_res != hipSuccess) {
        std::cerr << "Failed in HIP call: " << #hip_call \
                  << " at " << __FILE__ << ":" << __LINE__ \
                  << " with error: " << hipGetErrorString(hip_res) << std::e
        std::abort();
    }
}

__global__ void add_bfloat16(__hip_bfloat16* in1, __hip_bfloat16* in2, float
    int idx = threadIdx.x;
    if (idx < size) {
        // Load as bfloat16, perform addition, convert to float for output
        __hip_bfloat16 sum = in1[idx] + in2[idx];
        out[idx] = __bfloat162float(sum);
    }
}

int main() {
    constexpr size_t size = 32;
    constexpr float tolerance = 1e-1f;  // Allowable numerical difference

    // Initialize input vectors as floats
    std::vector<float> in1(size), in2(size);
    for (size_t i = 0; i < size; i++) {
        in1[i] = i + 0.5f;
        in2[i] = i + 0.5f;
    }

    // Compute expected results in full precision on CPU
    std::vector<float> cpu_out(size);
    for (size_t i = 0; i < size; i++) {
        cpu_out[i] = in1[i] + in2[i];  // Direct float addition
    }

    // Allocate device memory (store input as bfloat16, output as float)
    __hip_bfloat16 *d_in1, *d_in2;
    float *d_out;
    hip_check(hipMalloc(&d_in1, sizeof(__hip_bfloat16) * size));
    hip_check(hipMalloc(&d_in2, sizeof(__hip_bfloat16) * size));
    hip_check(hipMalloc(&d_out, sizeof(float) * size));

    // Convert input to bfloat16 and copy to device
    std::vector<__hip_bfloat16> in1_bf16(size), in2_b
    for (size_t i = 0; i < size; i++) {
        in1_bf16[i] = __float2bfloat16(in1[i]);
```

docs-7.2.1

```
        in2_bf16[i] = __float2bfloat16(in2[i]);
    }

    hip_check(hipMemcpy(d_in1, in1_bf16.data(), sizeof(__hip_bfloat16) * siz
    hip_check(hipMemcpy(d_in2, in2_bf16.data(), sizeof(__hip_bfloat16) * siz

    // Launch kernel
    add_bfloat16<<<1, size>>>(d_in1, d_in2, d_out, size);

    // Copy result back to host
    std::vector<float> gpu_out(size, 0.0f);
    hip_check(hipMemcpy(gpu_out.data(), d_out, sizeof(float) * size, hipMemc

    // Free device memory
    hip_check(hipFree(d_in1));
    hip_check(hipFree(d_in2));
    hip_check(hipFree(d_out));

    // Validation with tolerance
    for (size_t i = 0; i < size; i++) {
        if (std::fabs(cpu_out[i] - gpu_out[i]) > tolerance) {
            std::cerr << "Mismatch at index " << i << ": CPU result = " << c
                      << ", GPU result = " << gpu_out[i] << std::endl;
            std::abort();
        }
    }

    std::cout << "Success: CPU and GPU bfloat16 addition match within tolera

    return 0;
}
```

C++ Style Classes

`bfloat16` numbers can be used with C++ style classes:

```
__hip_bfloat16 bf16_val(1.1f);   // bfloat16
```

Vector Support

There is support for vectors of bfloat16 types:

- `__hip_bfloat162`: holds 2 values of bfloat16 numbers

docs-7.2.1

HIP Extensions

HIP also provides some extensions APIs for microscaling formats. These are supported on AMD GPUs. `gfx950` provides hardware acceleration for hip extensions. In fact most APIs are 1 to 1 mapping of hardware instruction.

Scale is also an input to the APIs. Scale is defined as type `__amd_scale_t` and is of format E8M0.

hipExt Types

hipExt microscaling APIs introduce a bunch of types which are used throughout the set of APIs.

Types

Types	Notes
`__amd_scale_t`	Store scale type which stores a value of E8M0.
`__amd_fp8_storage_t`	Store a single fp8 value.
`__amd_fp8x2_storage_t`	Store 2 packed fp8 value.
`__amd_fp8x8_storage_t`	Store 8 packed fp8 value.
`__amd_fp4x2_storage_t`	Store 2 packed fp4 value.
`__amd_fp4x8_storage_t`	Store 8 packed fp4 value.
`__amd_bf16_storage_t`	Store a single bf16 value.
`__amd_bf16x2_storage_t`	Store 2 packed bf16 value.
`__amd_bf16x8_storage_t`	Store 8 packed bf16 value.
`__amd_bf16x32_storage_t`	Store 32 packed bf16 value.
`__amd_fp16_storage_t`	Store a single fp16 value.
`__amd_fp16x2_storage_t`	Store 2 packed fp16 value.
`__amd_fp16x8_storage_t`	Store 8 packed fp16 value.
`__amd_fp16x32_storage_t`	Store 32 packed fp16 value.
`__amd_floatx2_storage_t`	Store 2 packed float value.
`__amd_floatx8_storage_t`	Store 8 packed float value.
`__amd_floatx16_storage_t`	Store 16 packed float value.
`__amd_floatx32_storage_t`	Store 32 packed float value.
`__amd_fp6x32_storage_t`	Store 32 packed fp6 value.
`__amd_shortx2_storage_t`	Store 2 packed short value.

docs-7.2.1

C-APIs

The naming style of C API is as follows:

All APIs start with `__amd`. `_`: is used as a separator. `cvt`: means convert i.e. convert from one format to another. `sr`: if an API name has **sr** in it, means it will do stochastic rounding and will expect an input as seed. `scale`: if an API has scale in it, means it will scale the values based on the `__amd_scale_t` input.

`create`: The following APIs will be used to create composite types from smaller values
`extract`: The following set of APIs will extract out individual values from a composite type.

Example: `__amd_cvt_fp8x8_to_bf16x8_scale` : this API converts 8-packed fp8 values to 8 packed bf16. This will also accept input of scale to do the conversion.

`__amd_extract_fp8x2` : this API will extract out a 2 packed fp8 value from 8 packed fp8 value based on index. Example of 8-packed fp8: `{a:{fp8, fp8}, b:{fp8, fp8}, c:{fp8, fp8}, d:{fp8, fp8}}` based on index 0, 1, 2 or 3 the API will return `a`, `b`, `c` or `d` respectively. `__amd_create_fp8x8` : this API will create 8 packed fp8 value from 4 inputs of 2 packed fp8 values.

C APIs

API	Notes
`float __amd_cvt_fp8_to_float(const __amd_fp8_storage_t, const __amd_fp8_interpretation_t)`	Convert a fp8 value to float.
`__amd_fp8_storage_t __amd_cvt_float_to_fp8_sr(const float, const __amd_fp8_interpretation_t, const unsigned int /* sr seed */)`	Convert a float to fp8 value with stochastic rounding, seed is passed as unsigned int argument.
`float __amd_cvt_fp8_to_float_scale(const __amd_fp8_storage_t, const __amd_fp8_interpretation_t, const __amd_scale_t)`	Convert a fp8 value to float with scale.
`float __amd_cvt_fp8_to_float_scale(const __amd_fp8_storage_t, const __amd_fp8_interpretation_t, const __amd_scale_t)`	Convert a fp8 value to float with scale.
`__amd_floatx2_storage_t __amd_cvt_fp8x2_to_floatx2(const __amd_fp8x2_storage_t, const __amd_fp8_interpretation_t)`	Convert 2 packed fp8 value to 2 packed float.
`__amd_fp8x2_storage_t __amd_cvt_floatx2_to_fp8x2(const __amd_floatx2_storage_t, const __amd_fp8_interpretation_t)`	Convert 2 packed float value to 2 packed fp8.
`__amd_fp4x2_storage_t __amd_cvt_floatx2_to_fp4x2_sr_scale(const __amd_floatx2_storage_t, const __amd_fp4_interpretation_t, const unsigned int /* sr seed */, const __amd_scale_t)`	Convert 2 packed float value to 2 packed fp4 with stochastic rounding and scale.
`__amd_floatx2_storage_t __amd_cvt_fp4x2_to_floatx2_scale(const __amd_fp4x2_storage_t , const`	docs-7.2.1 fp4 value to 2 packed float with

API	Notes
`__amd_fp4_interpretation_t, const __amd_scale_t)`	scale.
`__amd_fp4x2_storage_t __amd_cvt_floatx2_to_fp4x2_scale(const __amd_floatx2_storage_t, const __amd_fp4_interpretation_t, const __amd_scale_t)`	Convert 2 packed float value to 2 packed fp4 with scale.
`__amd_floatx2_storage_t __amd_cvt_fp8x2_to_floatx2_scale(const __amd_fp8x2_storage_t, const __amd_fp8_interpretation_t, const __amd_scale_t)`	Convert 2 packed fp8 value to 2 packed float with scale.
`__amd_fp8x2_storage_t __amd_cvt_floatx2_to_fp8x2_scale(const __amd_floatx2_storage_t, const __amd_fp8_interpretation_t, const __amd_scale_t)`	Convert 2 packed float value to 2 packed fp8 with scale.
`__amd_fp6x32_storage_t __amd_cvt_bf16x32_to_fp6x32_scale(const __amd_bf16x32_storage_t, const __amd_fp6_interpretation_t, const __amd_scale_t)`	Convert 32 packed bf16 value to 32 packed fp6 with scale.
`__amd_fp6x32_storage_t __amd_cvt_fp16x32_to_fp6x32_scale(const __amd_fp16x32_storage_t, const __amd_fp6_interpretation_t, const __amd_scale_t)`	Convert 32 packed fp16 value to 32 packed fp6 with scale.
`__amd_fp16x2_storage_t __amd_cvt_fp8x2_to_fp16x2_scale(const __amd_fp8x2_storage_t, const __amd_fp8_interpretation_t, const __amd_scale_t)`	Convert 2 packed fp8 value to 2 packed fp16 with scale.
`__amd_fp16x8_storage_t __amd_cvt_fp8x8_to_fp16x8_scale(const __amd_fp8x8_storage_t, const __amd_fp8_interpretation_t, const __amd_scale_t)`	Convert 8 packed fp8 value to 8 scale.

docs-7.2.1

API	Notes
`__amd_bf16x2_storage_t __amd_cvt_fp8x2_to_bf16x2_scale(const __amd_fp8x2_storage_t, const __amd_fp8_interpretation_t, const __amd_scale_t)`	Convert 2 packed fp8 value to 2 packed bf16 with scale.
`__amd_bf16x2_storage_t __amd_cvt_fp8x2_to_bf16x2_scale(const __amd_fp8x2_storage_t, const __amd_fp8_interpretation_t, const __amd_scale_t)`	Convert 2 packed fp8 value to 2 packed bf16 with scale.
`__amd_bf16x8_storage_t __amd_cvt_fp8x8_to_bf16x8_scale(const __amd_fp8x8_storage_t, const __amd_fp8_interpretation_t, const __amd_scale_t)`	Convert 8 packed fp8 value to 8 packed bf16 with scale.
`__amd_fp16x32_storage_t __amd_cvt_fp6x32_to_fp16x32_scale(const __amd_fp6x32_storage_t, const __amd_fp6_interpretation_t, const __amd_scale_t)`	Convert 32 packed fp6 value to 32 packed fp16 with scale.
`__amd_bf16x32_storage_t __amd_cvt_fp6x32_to_bf16x32_scale(const __amd_fp6x32_storage_t, const __amd_fp6_interpretation_t, const __amd_scale_t)`	Convert 32 packed fp6 value to 32 packed bf16 with scale.
`__amd_floatx32_storage_t __amd_cvt_fp6x32_to_floatx32_scale(const __amd_fp6x32_storage_t, const __amd_fp6_interpretation_t, const __amd_scale_t)`	Convert 32 packed fp6 value to 32 packed float with scale.
`__amd_fp16x2_storage_t __amd_cvt_fp4x2_to_fp16x2_scale(const __amd_fp4x2_storage_t, const __amd_fp4_interpretation_t, const __amd_scale_t)`	Convert 2 packed fp4 value to 2 packed fp16 with
`__amd_fp16x8_storage_t __amd_cvt_fp4x8_to_fp16x8_scale(const`	Convert 8 packed fp4 value to 8

docs-7.2.1

API	Notes
`__amd_fp4x8_storage_t, const __amd_fp4_interpretation_t, const __amd_scale_t)`	packed fp16 with scale.
`__amd_bf16x2_storage_t __amd_cvt_fp4x2_to_bf16x2_scale(const __amd_fp4x2_storage_t, const __amd_fp4_interpretation_t, const __amd_scale_t)`	Convert 2 packed fp4 value to 2 packed bf16 with scale.
`__amd_bf16x8_storage_t __amd_cvt_fp4x8_to_bf16x8_scale(const __amd_fp4x8_storage_t, const __amd_fp4_interpretation_t, const __amd_scale_t)`	Convert 8 packed fp4 value to 8 packed bf16 with scale.
`__amd_floatx8_storage_t __amd_cvt_fp4x8_to_floatx8_scale(const __amd_fp4x8_storage_t, const __amd_fp4_interpretation_t, const __amd_scale_t)`	Convert 8 packed fp4 value to 8 packed float with scale.
`__amd_fp4x8_storage_t __amd_cvt_floatx8_to_fp4x8_scale(const __amd_floatx8_storage_t, const __amd_fp4_interpretation_t, const __amd_scale_t)`	Convert 8 packed float value to 8 packed fp4 with scale.
`__amd_fp8x2_storage_t __amd_cvt_fp16x2_to_fp8x2_scale(const __amd_fp16x2_storage_t, const __amd_fp8_interpretation_t, const __amd_scale_t)`	Convert 2 packed fp16 value to 2 packed fp8 with scale.
`__amd_fp8x2_storage_t __amd_cvt_bf16x2_to_fp8x2_scale(const __amd_bf16x2_storage_t, const __amd_fp8_interpretation_t, const __amd_scale_t)`	Convert 2 packed bf16 value to 2 packed fp8 with scale.
`__amd_fp8x8_storage_t __amd_cvt_bf16x8_to_fp8x8_scale(const __amd_bf16x8_storage_t, const __amd_fp8_interpretation_t, const __amd_scale_t)`	Convert 8 packed bf16 value to 8 packed fp8 with scale.

docs-7.2.1

API	Notes
`__amd_floatx8_storage_t __amd_cvt_fp8x8_to_floatx8_scale(const __amd_fp8x8_storage_t, const __amd_fp8_interpretation_t, const __amd_scale_t)`	Convert 8 packed fp8 value to 8 packed float with scale.
`__amd_fp16_storage_t __amd_cvt_fp8_to_fp16_scale(const __amd_fp8_storage_t, const __amd_fp8_interpretation_t, const __amd_scale_t)`	Convert a fp8 value to fp16 with scale.
`__amd_bf16_storage_t __amd_cvt_fp8_to_bf16_scale(const __amd_fp8_storage_t, const __amd_fp8_interpretation_t, const __amd_scale_t)`	Convert a fp8 value to bf16 with scale.
`__amd_fp6x32_storage_t __amd_cvt_floatx16_floatx16_to_fp6x32_scale(const __amd_floatx16_storage_t, const __amd_floatx16_storage_t, const __amd_fp6_interpretation_t, const __amd_scale_t)`	Convert 2 inputs of 16-packed float values to 32 packed fp6 with scale.
`__amd_fp6x32_storage_t __amd_cvt_floatx32_to_fp6x32_scale(const __amd_floatx32_storage_t, const __amd_fp6_interpretation_t, const __amd_scale_t)`	Convert 32 packed float values to 32 packed fp6 with scale.
`__amd_fp6x32_storage_t __amd_cvt_floatx32_to_fp6x32_sr_scale(const __amd_floatx32_storage_t, const __amd_fp6_interpretation_t, const unsigned int, const __amd_scale_t)`	Convert 32 packed float values to 32 packed fp6 with stochastic rounding and scale.
`__amd_fp16_storage_t __amd_cvt_float_to_fp16_sr(const float, const unsigned int)`	Convert a float value to fp16 with stochastic rounding.

API	Notes
`__amd_fp16x2_storage_t __amd_cvt_float_float_to_fp16x2_sr(const float, const float, const unsigned int)`	Convert two inputs of float to 2 packed fp16 with stochastic rounding.
`__amd_bf16_storage_t __amd_cvt_float_to_bf16_sr(const float, const unsigned int)`	Convert a float value to bf16 with stochastic rounding.
`__amd_fp6x32_storage_t __amd_cvt_fp16x32_to_fp6x32_sr_scale(const __amd_fp16x32_storage_t, const __amd_fp6_interpretation_t, const unsigned int, const __amd_scale_t)`	Convert 32 packed fp16 values to 32 packed fp6 with stochastic rounding and scale.
`__amd_fp6x32_storage_t __amd_cvt_bf16x32_to_fp6x32_sr_scale(const __amd_bf16x32_storage_t, const __amd_fp6_interpretation_t, const unsigned int, const __amd_scale_t)`	Convert 32 packed bf16 values to 32 packed fp6 with stochastic rounding and scale.
`__amd_fp4x2_storage_t __amd_cvt_bf16x2_to_fp4x2_scale(const __amd_bf16x2_storage_t, const __amd_fp4_interpretation_t, const __amd_scale_t)`	Convert 2 packed bf16 value to 2 packed fp4 with scale.
`__amd_fp4x8_storage_t __amd_cvt_bf16x8_to_fp4x8_scale(const __amd_bf16x8_storage_t, const __amd_fp4_interpretation_t, const __amd_scale_t)`	Convert 8 packed bf16 value to 8 packed fp4 with scale.
`__amd_fp4x2_storage_t __amd_cvt_fp16x2_to_fp4x2_scale(const __amd_fp16x2_storage_t, const __amd_fp4_interpretation_t, const __amd_scale_t)`	Convert 2 packed fp16 value to 2

docs-7.2.1

API	Notes
`__amd_fp4x8_storage_t __amd_cvt_fp16x8_to_fp4x8_scale(const __amd_fp16x8_storage_t, const __amd_fp4_interpretation_t, const __amd_scale_t)`	Convert 8 packed fp16 value to 8 packed fp4 with scale.
`__amd_fp4x8_storage_t __amd_cvt_floatx8_to_fp4x8_sr_scale(const __amd_floatx8_storage_t, const __amd_fp4_interpretation_t, const unsigned int, const __amd_scale_t)`	Convert 8 packed float values to 8 packed fp4 with stochastic rounding and scale.
`__amd_fp4x2_storage_t __amd_cvt_bf16x2_to_fp4x2_sr_scale(const __amd_bf16x2_storage_t, const __amd_fp4_interpretation_t, const unsigned int,const __amd_scale_t)`	Convert 2 packed bf16 value to 2 packed fp4 with stochastic rounding and scale.
`__amd_fp4x8_storage_t __amd_cvt_bf16x8_to_fp4x8_sr_scale(const __amd_bf16x8_storage_t, const __amd_fp4_interpretation_t, const unsigned int, const __amd_scale_t)`	Convert 8 packed bf16 value to 8 packed fp4 with stochastic rounding and scale.
`__amd_fp4x2_storage_t __amd_cvt_fp16x2_to_fp4x2_sr_scale(const __amd_fp16x2_storage_t, const __amd_fp4_interpretation_t, const unsigned int, const __amd_scale_t)`	Convert 2 packed fp16 value to 2 packed fp4 with stochastic rounding and scale.
`__amd_fp4x8_storage_t __amd_cvt_fp16x8_to_fp4x8_sr_scale(const __amd_fp16x8_storage_t , const __amd_fp4_interpretation_t, const unsigned int, const __amd_scale_t)`	Convert 8 packed fp16 values to 8 packed fp4 with stochastic rounding

docs-7.2.1

API	Notes
`__amd_fp8x8_storage_t __amd_cvt_floatx8_to_fp8x8_sr_scale(const __amd_floatx8_storage_t, const __amd_fp8_interpretation_t, const unsigned int, const __amd_scale_t)`	Convert 8 packed float values to 8 packed fp8 with stochastic rounding and scale.
`__amd_fp8_storage_t __amd_cvt_fp16_to_fp8_sr_scale(const __amd_fp16_storage_t, const __amd_fp8_interpretation_t, const unsigned int, const __amd_scale_t)`	Convert a fp16 value to fp8 with stochastic rounding and scale.
`__amd_fp8x8_storage_t __amd_cvt_fp16x8_to_fp8x8_sr_scale(const __amd_fp16x8_storage_t, const __amd_fp8_interpretation_t, const unsigned int, const __amd_scale_t)`	Convert 8 packed fp16 values to 8 packed fp8 with stochastic rounding and scale.
`__amd_fp8_storage_t __amd_cvt_bf16_to_fp8_sr_scale(const __amd_bf16_storage_t, const __amd_fp8_interpretation_t, const unsigned int, const __amd_scale_t)`	Convert a bf16 value to fp8 with stochastic rounding and scale.
`__amd_fp8x8_storage_t __amd_cvt_bf16x8_to_fp8x8_sr_scale(const __amd_bf16x8_storage_t, const __amd_fp8_interpretation_t, const unsigned int, const __amd_scale_t)`	Convert 8 packed bf16 values to 8 packed fp8 with stochastic rounding and scale.
`__amd_fp16_storage_t __amd_cvt_fp8_to_fp16(const __amd_fp8_storage_t, const __amd_fp8_interpretation_t)`	Convert a fp8 value to fp16.
`__amd_fp16x2_storage_t __amd_cvt_fp8x2_to_fp16x2(const __amd_fp8x2_storage_t, const __amd_fp8_interpretation_t)`	Convert 2 packed fp8 value to 2
`__amd_fp8x2_storage_t __amd_cvt_fp16x2_to_fp8x2(const __amd_fp16x2_storage_t, const`	Convert 2 packed fp16 value to 2

docs-7.2.1

API	Notes
`__amd_fp8_interpretation_t)`	packed fp8.
`__amd_fp8x8_storage_t __amd_cvt_fp16x8_to_fp8x8_scale(const __amd_fp16x8_storage_t, const __amd_fp8_interpretation_t, const __amd_scale_t)`	Convert 8 packed fp16 values to 8 packed fp8 with scale.
`__amd_fp8x8_storage_t __amd_cvt_floatx8_to_fp8x8_scale(const __amd_floatx8_storage_t, const __amd_fp8_interpretation_t, const __amd_scale_t)`	Convert 8 packed float values to 8 packed fp8 with scale.
`__amd_fp8_storage_t __amd_cvt_fp16_to_fp8_sr(const __amd_fp16_storage_t, const __amd_fp8_interpretation_t, const short)`	Convert a fp16 value to fp8 with stochastic rounding.
`float2 __amd_cvt_floatx2_to_float2(const __amd_floatx2_storage_t)`	Convert 2 packed float value to hip's float2 type.
`__half __amd_cvt_fp16_to_half(const __amd_fp16_storage_t)`	Convert fp16 type to hip's __half type.
`__half2 __amd_cvt_fp16x2_to_half2(const __amd_fp16x2_storage_t)`	Convert 2 packed fp16 type to hip's __half2 type.
`__amd_fp16_storage_t __amd_cvt_half_to_fp16(const __half)`	Convert hip's __half type to fp16 type.
`__amd_fp16x2_storage_t __amd_cvt_half2_to_fp16x2(const __half2)`	Convert hip's __half2 type to 2 packed fp16.
`__hip_bfloat16 __amd_cvt_bf16_to_hipbf16(const __amd_bf16_storage_t)`	Convert bf16 type to

docs-7.2.1

API	Notes
`__hip_bfloat162 __amd_cvt_bf16x2_to_hipbf162(const __amd_bf16x2_storage_t)`	Convert 2 packed bf16 type to __hip_bfloat162 type.
`__amd_bf16_storage_t __amd_cvt_hipbf16_to_bf16(const __hip_bfloat16)`	Convert __hip_bfloat16 to bf16 type.
`__amd_bf16x2_storage_t __amd_cvt_hipbf162_to_bf16x2(const __hip_bfloat162)`	Convert __hip_bfloat162 to 2 packed bf16 type.

HIP EXT C++ API

There are C++ data structures also available. These are different from one in `<hip/hip_fp8.h>` header. These APIs expose a wider capability set which are exclusive to `gfx950`.

HIP EXT FP8 E4M3:

```
struct __hipext_ocp_fp8_e4m3  {
  // Constructor
  __host__ __device__ __hipext_ocp_fp8_e4m3(const float); // Create fp8 e4m3
  __host__ __device__ __hipext_ocp_fp8_e4m3(const float, const unsigned int
  __host__ __device__ __hipext_ocp_fp8_e4m3(const float, const unsigned int
  __host__ __device__ __hipext_ocp_fp8_e4m3(const __amd_fp16_storage_t, cons
  __host__ __device__ __hipext_ocp_fp8_e4m3(const __amd_bf16_storage_t, cons

  // Getters
  __host__ __device__ __amd_fp16_storage_t get_scaled_fp16(const __amd_scale
  __host__ __device__ __amd_bf16_storage_t get_scaled_bf16(const __amd_scale
  __host__ __device__ float get_scaled_float(const __amd_scale_t /* scale */

  // Operators
  __host__ __device__ operator float() const; // get
};
```

docs-7.2.1

HIP EXT FP8 E5M2:

```
struct __hipext_ocp_fp8_e5m2  {
  // Constructor
  __host__ __device__ __hipext_ocp_fp8_e5m2(const float); // Create fp8 e4m3
  __host__ __device__ __hipext_ocp_fp8_e5m2(const float, const unsigned int
  __host__ __device__ __hipext_ocp_fp8_e5m2(const float, const unsigned int
  __host__ __device__ __hipext_ocp_fp8_e5m2(const __amd_fp16_storage_t, cons
  __host__ __device__ __hipext_ocp_fp8_e5m2(const __amd_bf16_storage_t, cons

  // Getters
  __host__ __device__ __amd_fp16_storage_t get_scaled_fp16(const __amd_scale
  __host__ __device__ __amd_bf16_storage_t get_scaled_bf16(const __amd_scale
  __host__ __device__ float get_scaled_float(const __amd_scale_t /* scale */

  // Operators
  __host__ __device__ operator float() const; // get a float value
};
```

HIP EXT 2 Packed FP8 E4M3

```
struct __hipext_ocp_fp8x2_e4m3 {
  __host__ __device__ __hipext_ocp_fp8x2_e4m3(const float, const float); //
  __host__ __device__ __hipext_ocp_fp8x2_e4m3(const __amd_floatx2_storage_t)
  __host__ __device__ __hipext_ocp_fp8x2_e4m3(const __amd_floatx2_storage_t,
  __host__ __device__ __hipext_ocp_fp8x2_e4m3(const __amd_fp16x2_storage_t,
  __host__ __device__ __hipext_ocp_fp8x2_e4m3(const __amd_bf16x2_storage_t,

  // Getters
  __host__ __device__ __amd_fp16x2_storage_t get_scaled_fp16x2(const __amd_s
  __host__ __device__ __amd_bf16x2_storage_t get_scaled_fp16x2(const __amd_s
  __host__ __device__ __amd_floatx2_storage_t get_scaled_floatx2(const __amd

  // Operators
  __host__ __device__ operator __amd_floatx2_storage_t() const; // Get 2 pac
};
```

HIP EXT 2 Packed FP8 E5M2

```
struct __hipext_ocp_fp8x2_e5m2 {
  __host__ __device__ __hipext_ocp_fp8x2_e5m2(const float, const float); //
  __host__ __device__ __hipext_ocp_fp8x2_e5m2(const __amd_floatx2_storage_t)
  __host__ __device__ __hipext_ocp_fp8x2_e5m2(const __amd_floatx2_storage_t,
  __host__ __device__ __hipext_ocp_fp8x2_e5m2(const __amd_fp16x2_storage_t,
  __host__ __device__ __hipext_ocp_fp8x2_e5m2(const __amd_bf16x2_storage_t,

  // Getters
  __host__ __device__ __amd_fp16x2_storage_t get_scaled_fp16x2(const __amd_s
  __host__ __device__ __amd_bf16x2_storage_t get_scaled_fp16x2(const __amd_s
  __host__ __device__ __amd_floatx2_storage_t get_scaled_floatx2(const __amd

  // Operators
  __host__ __device__ operator __amd_floatx2_storage_t() const; // Get 2 pac
};
```

HIP EXT 32 packed FP6 E2M3

```
struct __hipext_ocp_fp6x32_e2m3 {
  __host__ __device__ __hipext_ocp_fp6x32_e2m3(const __amd_floatx16_storage_
  __host__ __device__ __hipext_ocp_fp6x32_e2m3(const __amd_floatx32_storage_
  __host__ __device__ __hipext_ocp_fp6x32_e2m3(const __amd_fp16x32_storage_t
  __host__ __device__ __hipext_ocp_fp6x32_e2m3(const __amd_fp16x32_storage_t
  __host__ __device__ __hipext_ocp_fp6x32_e2m3(const __amd_bf16x32_storage_t
  __host__ __device__ __hipext_ocp_fp6x32_e2m3(const __amd_bf16x32_storage_t

  // Getters
  __host__ __device__ __amd_floatx32_storage_t get_scaled_floatx32(const __a
  __host__ __device__ __amd_fp16x32_storage_t get_scaled_fp16x32(const __amd
  __host__ __device__ __amd_bf16x32_storage_t get_scaled_bf16x32(const __amd
};
```

HIP EXT 32 packed FP6 E3M2

```
struct __hipext_ocp_fp6x32_e3m2 {
  __host__ __device__ __hipext_ocp_fp6x32_e3m2(const __amd_floatx16_storage_
  __host__ __device__ __hipext_ocp_fp6x32_e3m2(const __amd_floatx32_storage_
  __host__ __device__ __hipext_ocp_fp6x32_e3m2(const __amd_fp16x32_storage_t
  __host__ __device__ __hipext_ocp_fp6x32_e3m2(const __amd_fp16x32_storage_t
  __host__ __device__ __hipext_ocp_fp6x32_e3m2(const __amd_bf16x32_storage_t
  __host__ __device__ __hipext_ocp_fp6x32_e3m2(const __amd_bf16x32_storage_t

  // Getters
  __host__ __device__ __amd_floatx32_storage_t get_scaled_floatx32(const __a
  __host__ __device__ __amd_fp16x32_storage_t get_sca
  __host__ __device__ __amd_bf16x32_storage_t get_sca
};
```

docs-7.2.1

HIP EXT 2 packed FP4

```
struct __hipext_ocp_fp4x2_e2m1 {
__host__ __device__ __hipext_ocp_fp4x2_e2m1(const float, const float, const
__host__ __device__ __hipext_ocp_fp4x2_e2m1(const __amd_floatx2_storage_t, c
__host__ __device__ __hipext_ocp_fp4x2_e2m1(const __amd_bf16x2_storage_t, co
__host__ __device__ __hipext_ocp_fp4x2_e2m1(const __amd_fp16x2_storage_t, co
__host__ __device__ __hipext_ocp_fp4x2_e2m1(const __amd_floatx2_storage_t, c
__host__ __device__ __hipext_ocp_fp4x2_e2m1(const __amd_bf16x2_storage_t, co
__host__ __device__ __hipext_ocp_fp4x2_e2m1(const __amd_fp16x2_storage_t, co

// Getters
__host__ __device__ __amd_floatx2_storage_t get_scaled_floatx2(const __amd_s
__host__ __device__ __amd_fp16x2_storage_t get_scaled_fp16x2(const __amd_sca
__host__ __device__ __amd_bf16x2_storage_t get_scaled_bf16x2(const __amd_sca
};
```

Terms and Conditions

ROCm Licenses and Disclaimers

Privacy

Trademarks

Supply Chain Transparency

Fair and Open Competition

UK Tax Strategy

Cookie Policy

Cookie Settings

© 2026 Advanced Micro Devices, Inc